GMX · SECURITY REVIEW

GMX
Donation Contracts.

A security review of Donation Contracts.

CLIENT
GMX

PROJECT
Donation Contracts

AUDITORS
Andreas Zottl, Zdravko Hristov, Ferran Celades, Vlad Zotov, Michael Lett

AUDIT WINDOW
September 15 – October 1, 2026

FINAL REPORT
October 1, 2026

§ 00

Contents.

38 pages: overview, confidence ranking, scope, 25 findings, disclaimer, and about Guardian.

§ 01 **Overview** P. 03

§ 02 **Confidence Ranking** P. 04

§ 03 **Scope & Classifications** P. 05

Files in Scope P. 06

§ 04 **Findings** P. 07

Main Review · 18 findings P. 09

Remediation Review · 7 findings P. 28

§ 05 **Disclaimer** P. 37

§ 06 **About Guardian** P. 38

§ 01

Overview.

Guardian reviewed Donation Contracts across 2 review rounds. The final report includes 1 Medium, 6 Low, 18 Informational findings.

AUDIT FIRM
Guardian

CLIENT
GMX

PROJECT
Donation Contracts

PREPARED BY
Andreas Zottl, Zdravko Hristov, Ferran Celades, Vlad Zotov, Michael Lett

AUDIT WINDOW
September 15 – October 1, 2026

FINAL REPORT
October 1, 2026

REPOSITORY
https://github.com/gmx-io/gmx-synthetics-private

METHODOLOGY
Smart contract · Manual · Main Review · Remediation Review

MAIN REVIEW COMMIT
58d0d01112a7

REMEDIATION REVIEW COMMIT
0a2386bb63ad

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer's assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	**0**	0	0	0	0
High	**0**	0	0	0	0
Medium	**1**	0	0	0	1
Low	**6**	0	5	0	1
Informational	**18**	0	12	0	6
All severities	**25**	**0**	**17**	**0**	**8**

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Donation Contracts in `https://github.com/gmx-io/gmx-synthetics-private`. The scope includes 707 source lines across 3 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
MAIN REVIEW		
contracts/donation/GlvDonationHandler.sol	322	394
contracts/donation/MarketPoolDonationHandler.sol	231	287
REMEDIATION REVIEW		
contracts/donation/DonationBatcher.sol	57	88
contracts/donation/GlvDonationHandler.sol	33	434
contracts/donation/MarketPoolDonationHandler.sol	64	350

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in. Status is where each finding ended up; its own page says how.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Pool donations change the rate used for uncheckpointed borrowing fees	Business logic errors	MEDIUM	RESOLVED	p. 9
L-01	Donations leave pre-donation signed GLV valuations usable for share pricing	Oracle	LOW	ACKNOWLEDGED	p. 10
L-02	Changing donation period duration implicitly resets or reuses quota accounting	Configuration	LOW	RESOLVED	p. 11
L-03	JIT deposits capture unsynced GLV rewards	Out of Scope	LOW	ACKNOWLEDGED	p. 12
L-04	ConfigSyncer risk updates retroactively rewrite accrued borrowing fees	Out of Scope	LOW	ACKNOWLEDGED	p. 13
L-05	Swap-path markets are not checkpointed before swaps rewrite accrued borrowing fees	Out of scope	LOW	ACKNOWLEDGED	p. 14
L-06	Per-side PnL caps inflate aggregate-priced GLV NAV and enable cross-market over-redemption	Unexpected behavior	LOW	ACKNOWLEDGED	p. 15
I-01	Donation execution repeats invariant already guaranteed by earlier checks	Superfluous Code	INFORMATIONAL	RESOLVED	p. 16
I-02	Donation keeper role definitions are duplicated outside the shared role modules	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-03	Market pool donation documentation names the wrong token	Typo	INFORMATIONAL	RESOLVED	p. 18
I-04	Market pool donation configuration can enable donations for a disabled market	Validation	INFORMATIONAL	RESOLVED	p. 19
I-05	Market pool donations omit the independent USD deposit cap	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-06	Gross donation cap weakens MEV protection	MEV	INFORMATIONAL	RESOLVED	p. 21
I-07	GLV donations use optimistic price bounds	Informational	INFORMATIONAL	RESOLVED	p. 23
I-08	A negative GLV constituent blocks donations to every healthy constituent	liveness / recovery limitation	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-09	MarketPoolDonationHandler lacks pre-transfer backing check, unmasking pre-existing deficits	accounting	INFORMATIONAL	ACKNOWLEDGED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-10	Protocol-mandated dead bootstrap GM strands donations	griefing / permanent asset lock	INFORMATIONAL	RESOLVED	p. 26
I-11	Same-token pool rounding omits one atomic token unit	Rounding	INFORMATIONAL	ACKNOWLEDGED	p. 27
REMEDIATION REVIEW					
I-01	Disabling a donation configuration inconsistently resets its state	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-02	Donation bootstrap checks are not complete	Business logic errors	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-03	Donation quotas are not coupled to the withdrawal-fee MEV bound	Info	INFORMATIONAL	ACKNOWLEDGED	p. 31
I-04	Dust circulation makes the bootstrap donation guard economically ineffective	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 33
I-05	Donation handlers report zero supply inconsistently	Error	INFORMATIONAL	ACKNOWLEDGED	p. 34
I-06	Signed GLV prices can become stale within donation batches	Events	INFORMATIONAL	ACKNOWLEDGED	p. 35
I-07	`GlvDonationHandler` unnecessarily inherits `BaseHandler`	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 36

MEDIUM M-01 RESOLVED

Pool donations change the rate used for uncheckpointed borrowing fees

CATEGORY Business logic errors

LOCATION https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L225-L232

REVIEW Main Review

DESCRIPTION

`MarketPoolDonationHandler.donate()` increases the accounted pool amount without first checkpointing cumulative borrowing factors. `MarketUtils.getNextCumulativeBorrowingFactor()` calculates the entire interval since the previous update using a rate derived from the current pool liquidity. Increasing liquidity can therefore apply a lower rate to time preceding the donation.

Note that the same class of issue exists in the `SwapUtils.swap()` path, but it was previously reported as `IOU-2` in the [2023 Guardian Report](https://github.com/gmx-io/gmx-synthetics/blob/main/audits/guardian/2023-07-11_GMX_Synthetics.pdf#page=50).

RECOMMENDATION

Consider checkpointing the cumulative borrowing factor before changing the poolAmount.

The best way would be to invoke `updateCumulativeBorrowingFactor()` for both the long and short directions from `donation()`. However, this would require making `MarketPoolDonationHandler` inherit from `OracleModule` and it will also require providing prices on each donation.

RESOLUTION

Status: Resolved.

LOW L-01

ACKNOWLEDGED

Donations leave pre-donation signed GLV valuations usable for share pricing

CATEGORY	Oracle
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L259-L279
REVIEW	Main Review

DESCRIPTION

GlvDonationHandler synchronizes additional GM holdings without minting GLV shares. This intentionally increases assets per share, but it neither invalidates signed GLV valuation snapshots that predate the increase nor establishes a freshness condition for subsequent GLV operations.

When a signed GLV price is supplied, GlvUtils returns that price multiplied by current supply without valuing the current recorded holdings. That result is used in actual deposit minting and withdrawal conversion. A report that omits an authorized donation can consequently overstate the shares to mint for a deposit or understate a withdrawing holder's share value, subject to the request's minimum-output checks.

RECOMMENDATION

Be aware and document this behavior.

RESOLUTION

Status: Acknowledged.

LOW L-02

RESOLVED

Changing donation period duration implicitly resets or reuses quota accounting

CATEGORY	Configuration
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L156-L162 https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L136-L142
REVIEW	Main Review

DESCRIPTION

Both configuration setters replace donationPeriodDuration while leaving donation state unchanged. State records only a numeric periodId and donatedAmount. Execution and the getters subsequently compare the stored identifier with a quotient calculated using the new duration.

This makes the behavior inconsistent, depending on the `periodID`. If the new one matches the current `periodID`, the donated amount will be preserved after the update.

On the other hand, if the configuration is modified with the intention of keeping the same limits with different duration, the donated amount will be reset to 0.

There is also not guarantee that the `periodID` will be ever-increasing. This makes it possible to transition to a lower period from a higher one.

RECOMMENDATION

Decide what the intended behavior of this flow should be and either acknowledge the finding or apply the necessary changes.

RESOLUTION

Status: Resolved.

LOW **L-03** ACKNOWLEDGED

JIT deposits capture unsynced GLV rewards

CATEGORY	Out of Scope
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/glv/glvDeposit/ExecuteGlvDepositUtils.sol#L67-L87
REVIEW	Main Review

DESCRIPTION

`ExecuteGlvDepositUtils.executeGlvDeposit()` processes a depositor's GM before calling `GlvUtils.getGlvValue()`. A GM reward transferred directly to a GLV is absent from `GlvToken.tokenBalances()` until `GlvToken.syncTokenBalance()` runs. The stale `cache.glvValue` then reaches `GlvDepositCalc.getMintAmount()`, overminting GLV tokens. A JIT depositor can redeem part of that pre-existing reward, diluting incumbent holders.

RECOMMENDATION

Fixing this requires changes in the core GMX logic, so be aware of it and either document it or consider applying a fix separately.

PROOF OF CONCEPT https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/ffffd66a93ed5d8d3b58417a87897f429382da34/test/guardian/GlvDirectRewardJitCapture.ts

RESOLUTION

Status: Acknowledged.

LOW L-04

ACKNOWLEDGED

ConfigSyncer risk updates retroactively rewrite accrued borrowing fees

CATEGORY	Out of Scope
LOCATION	packages/gmx-synthetics-private/contracts/config/ConfigSyncer.sol:sync():L47-L121
REVIEW	Main Review

DESCRIPTION

ConfigSyncer permits updates to borrowing-curve parameters and forwards them to Config.setUint without first checkpointing cumulative borrowing under the prior curve. The next market action computes a rate from the new parameters and multiplies it by the entire interval since the last checkpoint. A legitimate rate reduction therefore erases fees accrued under the old curve, while an increase retroactively charges a rate that was not in force. RiskOracleConfig's separate update path explicitly settles borrowing before writing these keys, corroborating the missing checkpoint.

Impact: The passing production-fork PoC opens and closes a real $5 million position through the protocol handlers. Reducing the active kink factors after 30 uncheckpointed days, immediately before the identical close, increases the trader's actual payout by about 90.44 WETH ($224,556), directly reducing LP/protocol fee revenue. The error affects all positions on the updated side and can also overcharge or unexpectedly liquidate positions when the curve is increased.

RECOMMENDATION

Before ConfigSyncer writes any borrowing-rate key, atomically checkpoint the affected market and side using validated oracle prices under the old configuration, matching RiskOracleConfig._settleBorrowingIfRequired. Otherwise remove borrowing keys from the generic Config.setUint synchronization route.

PROOF OF CONCEPT https://github.com/GuardianOrg/SentryRepo-Donation-Contracts---Round-1-01a0a3f3547a78999f3770f65a947177/blob/refinery/poc-01a0a3f4-7335-7bb3-b5be-7e0c821f814e/test/refinery-poc/configsyncer-risk-updates-retroactively-rewrite-accrued-borr-beeb7d71.sol

RESOLUTION

Status: Acknowledged.

LOW L-05

ACKNOWLEDGED

Swap-path markets are not checkpointed before swaps rewrite accrued borrowing fees

CATEGORY	Out of scope
LOCATION	packages/gmx-synthetics-private/contracts/order/OrderUtils.sol:_createOrder():L294-L319
REVIEW	Main Review

DESCRIPTION

MarketSwap orders store `address(0)` as order.market and place their actual markets only in swapPath. `BaseOrderHandler` resolves `swapPathMarkets`, but `ExecuteOrderUtils` checkpoints funding and borrowing only for params.market, which is empty for a pure swap.
`SwapUtils` then changes the real path market's pool amounts. The next position action applies the post-swap utilization rate to the entire interval since the last checkpoint, allowing a user to add the charged-side token, crystallize a retroactively reduced cumulative borrowing factor, and reverse the swap.

The fork PoC uses public request/execution paths and authenticated reports, passes as submitted, and demonstrates a 499.549 WETH net benefit after the complete round trip.

RECOMMENDATION

A fix would checkpoint funding and cumulative borrowing for each distinct swapPath market using its pre-swap balances and validated prices, instead of relying on `params.market`, which is intentionally empty for `MarketSwap` orders.

PROOF OF CONCEPT https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/refinery-poc/test/refinery-poc/SwapPathMarketCheckpointPOC.t.sol

RESOLUTION

Status: Acknowledged.

LOW L-06

ACKNOWLEDGED

Per-side PnL caps inflate aggregate-priced GLV NAV and enable cross-market over-redemption

CATEGORY	Unexpected behavior
LOCATION	`packages/gmx-synthetics-private/contracts/market/MarketUtils.sol:getPoolValueInfo():L419-L454`
REVIEW	Main Review

DESCRIPTION

Market pool valuation caps each side's positive pending PnL separately before netting it against the opposite side's uncapped loss. An equal-sized long and short can therefore produce positive aggregate market value once the winning side exceeds its cap, although their true aggregate PnL is approximately zero.

The intentional per-side cap does not make this composed NAV result realizable: settling the solvent losing leg first credits its loss to the winning leg's payout-token pool, increases that leg's dynamic cap, and permits the winner to recover the formerly capped PnL. The pool therefore never realizes the marked surplus.

Aggregate-priced GLVs nevertheless include it, and a holder can redeem the inflated GLV value through another healthy constituent because execution validates only the selected withdrawal market's PnL.

The main impact of the finding is about the core GMX system, but `GlvDonationHandler` also uses the inflated aggregate value for its factor allowance.

RECOMMENDATION

Consider whether the impact shown in the POC - withdrawing value from constituent market due to another inflated market's valuation - needs to be mitigated.

If not, just document the issue.

PROOF OF CONCEPT `https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/refinery-poc/test/refinery-poc/PnLCappingPOC.t.sol`

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01 RESOLVED

Donation execution repeats invariant already guaranteed by earlier checks

CATEGORY	Superfluous Code
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L206-L208
REVIEW	Main Review

DESCRIPTION

Both donate functions reject disabled configurations before checking fundingAccount against zero. Their private configuration mappings can only be written through setters, and an enabled configuration cannot be stored with a zero funding account. The repeated zero-address branches are therefore unreachable under the current implementation.

RECOMMENDATION

Be aware and consider to remove superfluous code.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

ACKNOWLEDGED

Donation keeper role definitions are duplicated outside the shared role modules

CATEGORY	Best Practices
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L66 https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L109-L112 https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L55-L56 https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L92-L95
REVIEW	Main Review

DESCRIPTION

Both handlers define the same DONATION_KEEPER hash and onlyDonationKeeper modifier locally instead of using the shared Role/Role2 libraries and RoleModule.

RECOMMENDATION

Be aware and consider to define DONATION_KEEPER once in the shared role library and onlyDonationKeeper once in RoleModule.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03 RESOLVED

Market pool donation documentation names the wrong token

CATEGORY	Typo
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L162-L163
REVIEW	Main Review

DESCRIPTION

The donate comment says the function pulls the configured market token. However the market's pool token is pulled in reality.

RECOMMENDATION

Replace 'configured market token' with 'configured market's underlying pool token' in the donate documentation.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04 RESOLVED

Market pool donation configuration can enable donations for a disabled market

CATEGORY Validation

LOCATION https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L110-L117

REVIEW Main Review

DESCRIPTION

setMarketPoolDonationConfig verifies market existence and, when enabling donations, single-token eligibility and absence of virtual inventory. It does not reject a globally disabled market. In contrast, setGlvDonationConfig calls MarketUtils.validateEnabledMarket inside its isEnabled branch.

The pool setter can therefore store and emit an enabled donation configuration that cannot currently execute.

RECOMMENDATION

Be aware and for consistency with the GLV setter, consider to call MarketUtils.validateEnabledMarket(dataStore, market) inside the isEnabled branch of setMarketPoolDonationConfig.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-05 ACKNOWLEDGED

Market pool donations omit the independent USD deposit cap

CATEGORY	Validation
LOCATION	https://github.com/gmx-io/gmx-synthetics-private/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L225-L232
REVIEW	Main Review

DESCRIPTION

MarketPoolDonationHandler.donate increases poolAmount and calls validatePoolAmount, but omits the validatePoolUsdForDeposit check used for normal GM deposits.

RECOMMENDATION

Be aware and document this behavior or consider to enforce MarketUtils.validatePoolUsdForDeposit after updating poolAmount.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06 RESOLVED

Gross donation cap weakens MEV protection

CATEGORY	MEV
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/MarketPoolDonationHandler.sol#L189-L192
REVIEW	Main Review

DESCRIPTION

`MarketPoolDonationHandler.donate()` applies `maxDonationFactor` to the raw `poolAmount`, without respecting the traders max pnl cap. Part of the donated value may therefore be offset by newly recognized trader PnL rather than increasing GM value. Similarly, the newly donated amount may also increase the capped value of the positive price-impact received.

On the other hand, withdrawals are valued using the net pool value calculated by `MarketUtils.getPoolValueInfo()`, which accounts for trader PnL and the position impact pool. When net GM equity is below the gross pool value, a maximum-sized donation can increase the GM price by more than `maxDonationFactor`.

Consequently, setting `withdrawalFeeFactor` above `maxDonationFactor` does not by itself prove that an immediate deposit-donation-withdrawal strategy is unprofitable. With a `0.5%` donation factor and `4%` withdrawal fee, the strategy remains unprofitable only while net GM equity is at least approximately `12%` of gross pool value. This is likely true during ordinary operation but is not enforced.

RECOMMENDATION

Apply `maxDonationFactor` to the pre-donation net GM equity using the same valuation applied to withdrawals. Alternatively, document and enforce a minimum net-to-gross pool-value ratio with an appropriate safety margin.

Also document that every accepted pool donation immediately increases the pool value used by `capPositiveImpactUsdByPositionImpactPool()`, potentially increasing payouts for pre-donation positive-impact claims and recording the additional payout in `lentPositionImpactPoolAmount`.

PROOF OF CONCEPT https://github.com/GuardianOrg/SentryRepo-Donation-Contracts---Round-1-01a0a3f3547a78999f3770f65a947177/blob/refinery/poc-01a0a3f4-7335-7bb3-b5be-7e0c821f814e/test/refinery-poc/gross-donation-factor-may-exceed-net-gm-equity-movement-5e0030c2.sol

I-06 Gross donation cap weakens MEV protection CONTINUED

RESOLUTION

Status: Resolved.

INFORMATIONAL I-07 RESOLVED

GLV donations use optimistic price bounds

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L222-L241 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/58d0d01112a7b36a372e3351a45626f60098b874/contracts/donation/GlvDonationHandler.sol#L309-L327
REVIEW	Main Review

DESCRIPTION

`GlvDonationHandler.donate()` maximizes both `execution.glvValue` and `execution.marketPoolValue`. Maximizing `execution.glvValue` increases `maxDonationUsd`, making the proportional donation limit less conservative. Additionally, `_getMarketPoolValue()` checks only the market's maximized value, allowing a market with a negative minimized value to pass.

RECOMMENDATION

Calculate `execution.glvValue` with `maximize = false`. In `_getMarketPoolValue()`, call `getPoolValueInfo()` with `maximize = false` for the negative-value check, then return a second calculation using `maximize = true` for `donationUsd` and balance-limit validation.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08

ACKNOWLEDGED

A negative GLV constituent blocks donations to every healthy constituent

CATEGORY	liveness / recovery limitation
LOCATION	`packages/gmx-synthetics-private/contracts/donation/GlvDonationHandler.sol:donate():L187-L282`
REVIEW	Main Review

DESCRIPTION

GlvDonationHandler obtains the factor-cap denominator through the reverting GlvUtils.getGlvValue. Aggregate valuation aborts when any supported market with a nonzero GLV balance has negative pool value, so that one market prevents donations of unrelated healthy GM to the same GLV. The restriction persists until the market recovers or governance disables it, configures the residual-balance dust threshold, and removes it. The PoC demonstrates both the revert and this governance recovery path.

RECOMMENDATION

Be aware and document this behavior.

PROOF OF CONCEPT `https://github.com/GuardianOrg/SentryRepo-Donation-Contracts---Round-1-01a0a3f3547a78999f3770f65a947177/blob/refinery/poc-01a0a3f4-7335-7bb3-b5be-7e0c821f814e/test/refinery-poc/a-negative-glv-constituent-blocks-donations-to-every-healthy-fb0b590b.sol`

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09 ACKNOWLEDGED

MarketPoolDonationHandler lacks pre-transfer backing check, unmasking pre-existing deficits

CATEGORY	accounting
LOCATION	packages/gmx-synthetics-private/contracts/donation/MarketPoolDonationHandler.sol:donate():L164-L252
REVIEW	Main Review

DESCRIPTION

`MarketUtils.validateMarketTokenBalance()` reverts when the market has `balance < aggregate claimableFundingAmount` (documented late-liquidation edge in MarketUtils L3652-L3655). Because of this normal funding claims revert.

`MarketPoolDonationHandler.donate()` performs no pre-transfer solvency check. This makes it possible to donate to a market in the described state and make the `balance > claimableFundingAmount`, which enables claims again.

RECOMMENDATION

Be aware and document this behavior.

PROOF OF CONCEPT https://github.com/GuardianOrg/SentryRepo-Donation-Contracts---Round-1-01a0a3f3547a78999f3770f65a947177/blob/refinery/poc-01a0a3f4-7335-7bb3-b5be-7e0c821f814e/test/refinery-poc/marketpooldonationhandler-lacks-pre-transfer-backing-check-u-84b2dfa3.sol

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-10 RESOLVED

Protocol-mandated dead bootstrap GM strands donations

CATEGORY	griefing / permanent asset lock
LOCATION	packages/gmx-synthetics-private/contracts/donation/MarketPoolDonationHandler.sol:donate():L182-L232
REVIEW	Main Review

DESCRIPTION

donate() treats any nonzero MarketToken.totalSupply as proof that a redeemable beneficiary exists. GMX's configured first-deposit path, however, requires bootstrap GM to be minted to inaccessible address(1). After all ordinary LPs withdraw, totalSupply can consist solely of dead bootstrap shares. Donations still transfer approved underlying, increase accounted pool liquidity, and consume quota without minting any controllable GM. Later deposits are priced against the elevated pool and dead supply, so they acquire only a claim equal to their new contribution; they do not recover the prior donation.

Impact: A permissionless transition from a normally configured live market to address(1)-only supply can cause an honest scheduled keeper execution to strand funding assets through every normal user flow. The fork PoC creates a real market, performs the protocol-mandated bootstrap deposit, proves 100% of GM is at address(1), and executes two cap-valid donations that consume 3 WETH and grow dead backing from 1 to 4 WETH. It then deposits 1 WETH for a fresh live LP and withdraws all newly minted GM through the real withdrawal handler: exactly that 1 WETH returns while the entire original 4 WETH remains dead-backed. Repeated donation periods can therefore expand the irrecoverable minimum far beyond the bootstrap floor until privileged governance/controller recovery or migration occurs.

RECOMMENDATION

Be aware and document this behavior.

PROOF OF CONCEPT https://github.com/GuardianOrg/SentryRepo-Donation-Contracts---Round-1-01a0a3f3547a78999f3770f65a947177/blob/refinery/poc-01a0a3f4-7335-7bb3-b5be-7e0c821f814e/test/refinery-poc/protocol-mandated-dead-bootstrap-gm-strands-donations-ee77e05c.sol

RESOLUTION

Status: Resolved.

INFORMATIONAL I-11 ACKNOWLEDGED

Same-token pool rounding omits one atomic token unit

CATEGORY	Rounding
LOCATION	`contracts/market/MarketUtils.sol:getPoolValueInfo():L376-L382 contracts/market/MarketUtils.sol:getPoolAmount():L645-L651 contracts/donation/MarketPoolDonationHandler.sol:donate():L212-L232`
REVIEW	Main Review

DESCRIPTION

In same-token markets, the long and short sides share one stored pool amount. The amount is divided by two for each side, so when it is odd, integer rounding causes the two halves to add up to one atomic token unit less than the stored amount.

This can make a minimum-size donation temporarily absent from valuation, leave one unit behind after a full GM withdrawal, and understate the GLV per-market balance cap by the value of one unit. The discrepancy is limited to the token's smallest unit (for example, 1 wei of WETH) and is economically negligible.

RECOMMENDATION

Be aware and consider to handle same-token markets as one amount, or assign the division remainder to one side so the two sides always sum to the stored pool amount.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Disabling a donation configuration inconsistently resets its state

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/4d79fae0f89f740cbfcdd9a934e8ad298b0b3496/contracts/donation/MarketPoolDonationHandler.sol#L145-L157
REVIEW	Remediation Review

DESCRIPTION

When `setMarketPoolDonationConfig()` disables a configuration, the existing `marketPoolDonationStates` entry is deleted only if `donationPeriodDuration` changes. Consequently, disabling with the existing duration preserves the consumed quota, whereas disabling with a different duration resets it. Because disabled configurations do not validate these parameters, the resulting state-reset behavior depends on otherwise inactive configuration values.

RECOMMENDATION

This behavior may be desired if the team needs a way to disable a configuration without resetting its state. If that's the case, consider documenting it.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02 ACKNOWLEDGED

Donation bootstrap checks are not complete

CATEGORY	Business logic errors
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/MarketPoolDonationHandler.sol#L204-L210 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L217-L227
REVIEW	Remediation Review

DESCRIPTION

`MarketPoolDonationHandler._donate()` and `GlvDonationHandler.donate()` prevent donations when the entire GM or GLV supply is held by the inaccessible bootstrap receiver, `address(1)`. However, both checks only subtract the token balance held directly by `address(1)`.

For a multichain deposit, `ExecuteDepositUtils._executeDeposit()` and `ExecuteGlvDepositUtils.executeGlvDeposit()` mint the resulting GM or GLV tokens to the `MultichainVault` and record ownership through `Keys.multichainBalanceKey(receiver, token)`. Consequently, a first deposit whose required receiver is `address(1)` can produce a nonzero token supply while `balanceOf(address(1))` remains zero. The checked live Arbitrum and Avalanche deployments have no enabled source-chain IDs, and all multichain balances for `address(1)` across registered GM and GLV tokens were zero at review time.

There is also one more condition where all the shares of a GM vault are dead.

```solidity
gm.totalSupply() == gm.balanceOf(address(1)) + gm.balanceOf(glv) && glv.totalSupply() == glv.balanceOf(address(1))
```

`In that state, all GMs are held by one or more GLV vaults and all the glv shares are locked at address(1)`.

RECOMMENDATION

Before enabling multichain deposits, either document this interaction or include `dataStore.getUint(Keys.multichainBalanceKey(address(1), token))` in the bootstrap dead-supply calculation for both donation handlers.

Also consider whether a change is needed in order to account for GM held in dead GLV vaults.

I-02 Donation bootstrap checks are not complete CONTINUED

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03 ACKNOWLEDGED

Donation quotas are not coupled to the withdrawal-fee MEV bound

CATEGORY	Info
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/MarketPoolDonationHandler.sol#L216-L230 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L239-L281
REVIEW	Remediation Review

DESCRIPTION

`setMarketPoolDonationConfig()` configures `maxDonationFactor`, `donationPeriodDuration`, and `maxDonationAmountPerPeriod` independently of the market's withdrawal fee. During `_donate()`, `maxDonationFactor` limits each donation relative to the current pre-donation `poolValue`, while `maxDonationAmountPerPeriod` limits the cumulative token amount within a fixed Unix-time period.

The withdrawal fee is intended to prevent a just-in-time depositor from entering before a donation, receiving part of the resulting GM price increase, and withdrawing immediately afterward. For one maximum donation with factor `f` and withdrawal fee `w`, the position is unprofitable when:

```text
(1 + f) × (1 - w) ≤ 1
```

However, a depositor can hold the same GM position across multiple donations while paying the withdrawal fee only once. After `n` maximum donations, the corresponding condition becomes:

```text
(1 + f)ⁿ × (1 - w) ≤ 1
```

Equivalently, if `D` is the cumulative donated USD value and V_0 is the pool value when the depositor enters, the cumulative limit must satisfy:

```text
D ≤ V₀ × w / (1 - w)
```

`maxDonationAmountPerPeriod` is an absolute token-denominated limit and therefore does not automatically adjust when `poolValue`, the donation token price, or the withdrawal fee

I-03 Donation quotas are not coupled to the withdrawal-fee MEV bound CONTINUED

changes. Additionally, because donation periods reset at fixed boundaries, a position may capture donations immediately before and after a boundary while paying one withdrawal fee.

Permissioned donation execution and appropriately selected configuration values can prevent profitable JIT behavior. Nevertheless, the required relationship is an operational assumption that is not documented or enforced by the contracts.

The same invariant applies to GLV donations. GlvDonationHandler.donate() grants a new factor allowance for every donation and tracks period
> usage separately for each (glv, market) pair.

RECOMMENDATION

Document the required relationship between `maxDonationFactor`, `maxDonationAmountPerPeriod`, `donationPeriodDuration`.

When configuring donations, derive `maxDonationAmountPerPeriod` from a conservative minimum expected `poolValue` and maximum donation-token price such that cumulative donations during any expected deposit-to-withdrawal exposure window satisfy:

```text
D ≤ V₀ × w / (1 - w)
```

Account for donations across adjacent fixed-period boundaries when determining this limit.

In addition, note that data stream providers may have more than one valid price at a time. This makes it technically possible for a withdrawal to use older price than the donation logic.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

Dust circulation makes the bootstrap donation guard economically ineffective

CATEGORY	Warning
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/MarketPoolDonationHandler.sol#L203-L218 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L217-L266
REVIEW	Remediation Review

DESCRIPTION

`MarketPoolDonationHandler._donate()` and `GlvDonationHandler.donate()` reject a donation only when `totalSupply()` is no greater than the shares held directly by the inaccessible bootstrap receiver, `address(1)`. Once any positive circulating balance exists, however small, each handler permits a donation sized from the value backing the entire token supply, including the bootstrap shares.

RECOMMENDATION

If enforcing this onchain is desired, add a narrowly scoped minimum-circulation or maximum-dead-share-ratio configuration and scale the factor-based donation limit by `circulatingSupply / totalSupply` in both handlers after computing the effective dead supply.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05 ACKNOWLEDGED

Donation handlers report zero supply inconsistently

CATEGORY	Error
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/MarketPoolDonationHandler.sol#L203-L209 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L217-L226
REVIEW	Remediation Review

DESCRIPTION

`MarketPoolDonationHandler._donate()` previously reverted with `Errors.EmptyMarketTokenSupply()` when `marketTokenSupply` was zero. The remediation replaces that check with `marketTokenSupply <= deadSupply`, which reverts with `EmptyCirculatingMarketTokenSupply()`.

Because `balanceOf(address(1))` must also be zero when `totalSupply()` is zero, the replacement still prevents the donation. It no longer distinguishes a market with no issued GM from a market whose entire nonzero supply consists of inaccessible bootstrap shares.

Conversely, `GlvDonationHandler.donate()` retains the explicit `Errors.EmptyGlvTokenSupply()` check before applying its analogous `glvSupply <= deadSupply` check. The two parallel handlers therefore expose different error semantics for the same zero-supply state.

RECOMMENDATION

Either restore an explicit `marketTokenSupply == 0` check in `MarketPoolDonationHandler._donate()` or remove the one in `GlvDonationHandler`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06 ACKNOWLEDGED

Signed GLV prices can become stale within donation batches

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L236-L265 https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/DonationBatcher.sol#L64-L88
REVIEW	Remediation Review

DESCRIPTION

The code acknowledges that multiple donations to the same GLV within `DonationBatcher.multicall()` may reuse a signed GLV price that no longer reflects the GLV's state after the first donation.

The same behavior occurs when `MarketPoolDonationHandler.donate()` first donates to a GM market held by the GLV. The GM donation increases the value of the GLV's existing GM balance, but a subsequent `GlvDonationHandler.donate()` may still use the pre-batch signed GLV price through `GlvUtils.getGlvValue()`.

Consequently, `maxDonationUsd` is calculated conservatively and may reject an otherwise-valid donation, while the emitted `GlvDonation` and `GlvValueUpdated` values may understate the current GLV value.

RECOMMENDATION

Document both sources of intra-batch staleness and clarify that the emitted GLV value may not represent the final batch state.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07** ACKNOWLEDGED

`GlvDonationHandler` unnecessarily inherits `BaseHandler`

CATEGORY	Superfluous Code
LOCATION	https://github.com/GuardianOrg/mirror-gmx-io-gmx-synthetics-private-3a775f46e98e78cb95b39c3a3dc86bdc/blob/0a2386bb63ad843b13872e94015e931b966eeba0/contracts/donation/GlvDonationHandler.sol#L26
REVIEW	Remediation Review

DESCRIPTION

`GlvDonationHandler` inherits `BaseHandler` but does not use its request-cancellation, error-handling, data-length, or native-token functionality. In particular, the donation flow never unwraps WNT, making the inherited payable `receive()` entry point unnecessary.

RECOMMENDATION

Either acknowledge or follow the narrower inheritance pattern used by `MarketPoolDonationHandler`: inherit `RoleModule`, `GlobalReentrancyGuard`, and `OracleModule` directly, and declare and initialize `eventEmitter` locally.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

$45B+
DIGITAL ASSETS UNDER REVIEW

200+
ENGAGEMENTS COMPLETED

500+
CRITICAL & HIGH FINDINGS REPORTED

Reach us at audits@guardianaudits.com.